Digital Electronic Circuits (EC2L004)

Lecture 7:Gate-Level minimization-III

Learning Objectives

- Gate-level minimization
  - Don’t care conditions
- NAND & NOR implementation
- AND-OR-INVERT implementation
- OR-AND-INVERT implementation
- Exclusive-OR function

Don’t Care Conditions

- Functions that have unspecified output for some input combinations are called incompletely specified functions.

- Unspecified minterms of a functions are called ‘don’t care’ conditions. We simply *don’t care* whether the value of 0 or 1 is assigned to *F* for a particular minterm.

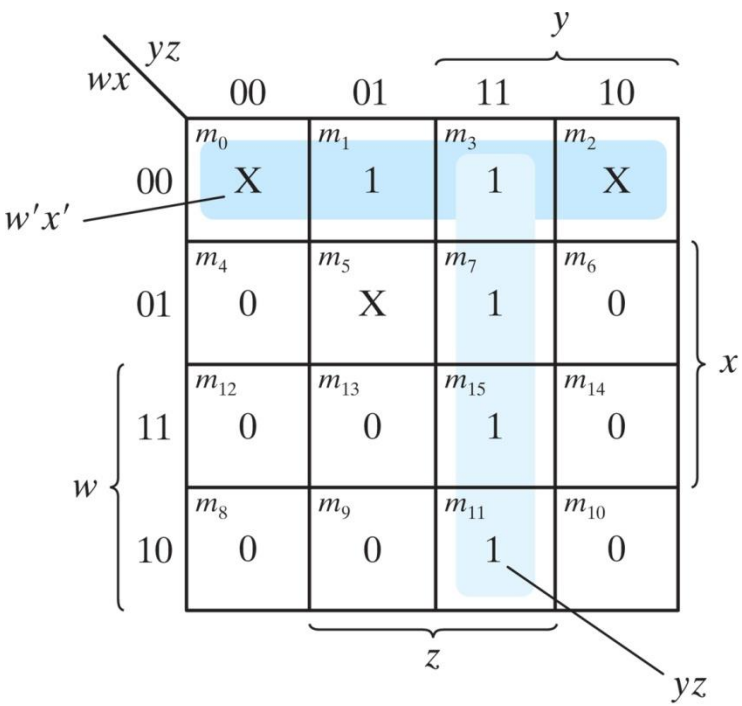
- Don’t care conditions are represented by *X* in the K-Map table.

  - *Don’t care* conditions play a central role in the specification and optimization of logic circuits as they represent the degrees of freedom of transforming a network into a functionally equivalent one.

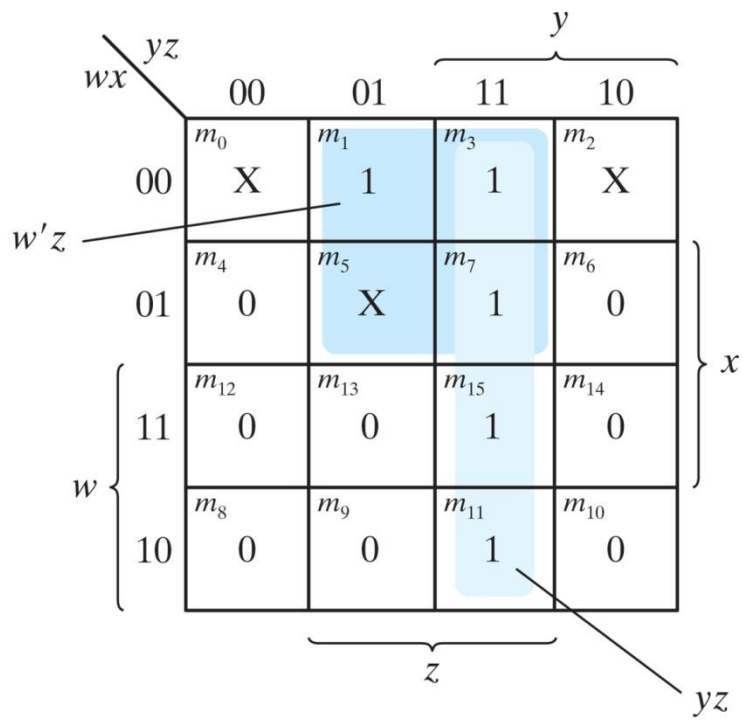
Don’t Care Conditions

- Example 18: Simplify the Boolean function

  $F(w, x, y, z) = \sum(1,3,7,11,15)$, $\sum_{d}(w, x, y, z) = \sum(0,2,5)$



(a) $F = yz + w'x'$

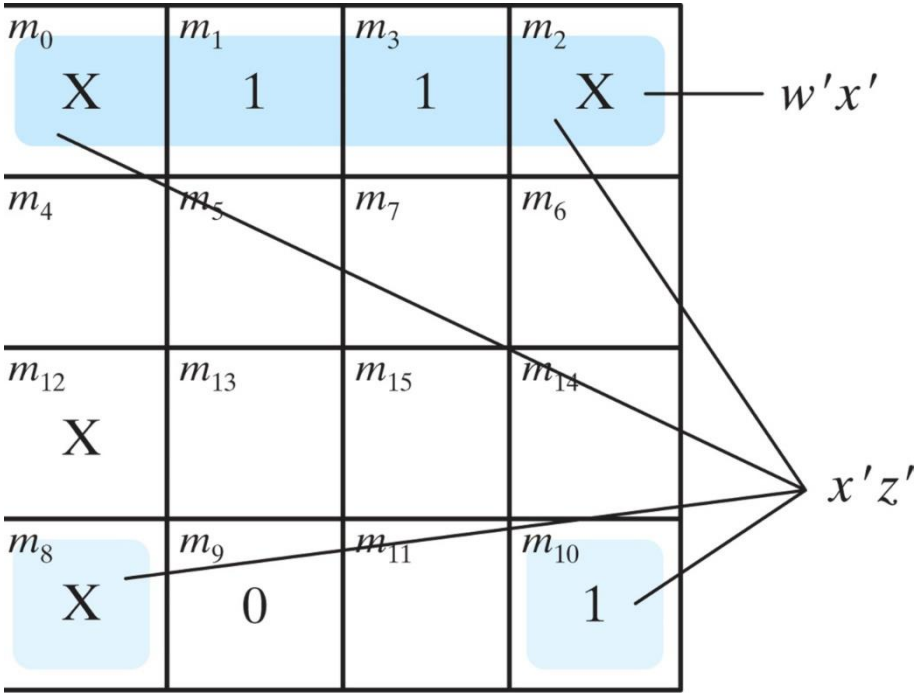


(b) $F = yz + w'z$

Don’t Care Conditions

- Example 19: Simplify the Boolean function

$$F(w, x, y, z) = \sum(1,3,10),\ \sum_d(w, x, y, z) = \sum(0,2,8,12)$$

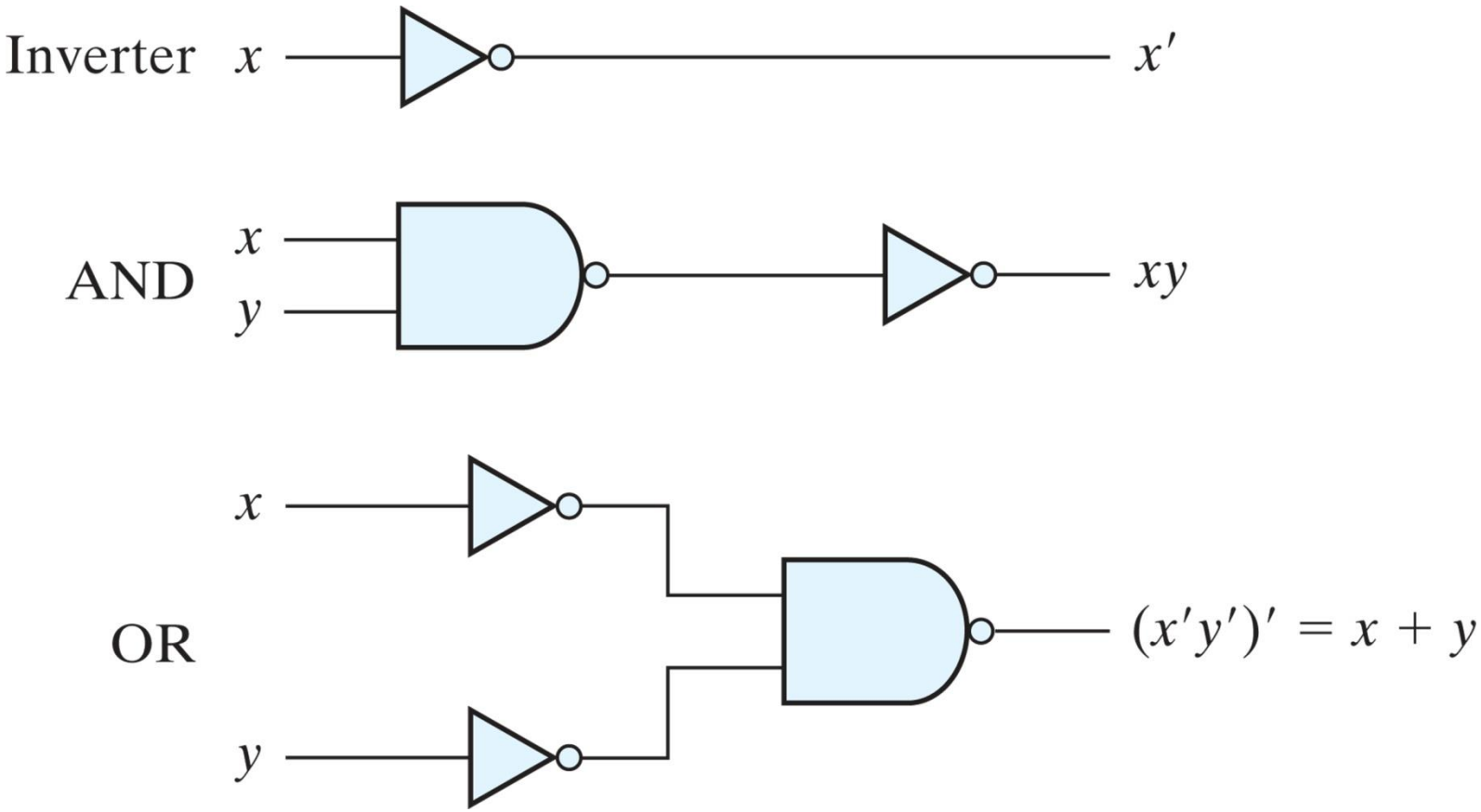


Nand and Nor implementation

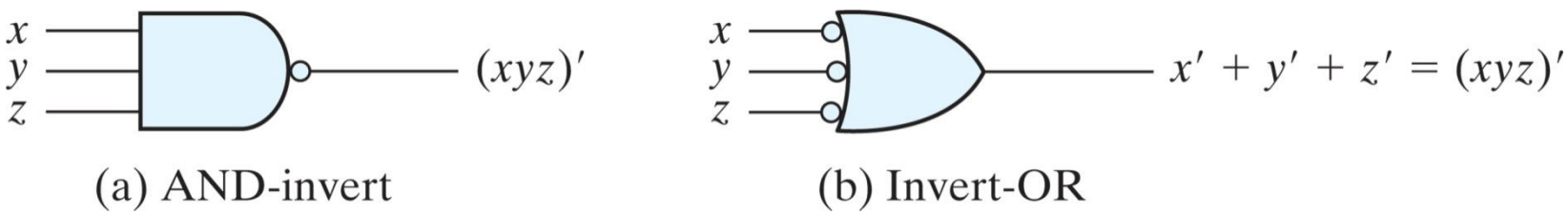
- Nand and Nor gates are easier to fabricate with electronic components and are the basic gates used in all IC digital logic families.
  - Universal gates---any logic circuit can be implemented with it.

- NAND Circuits
  - A convenient way to implement a Boolean function with NAND gates is to obtain the simplified Boolean function in terms of Boolean operators and then convert the function to NAND logic.

Logic operations with NAND gates



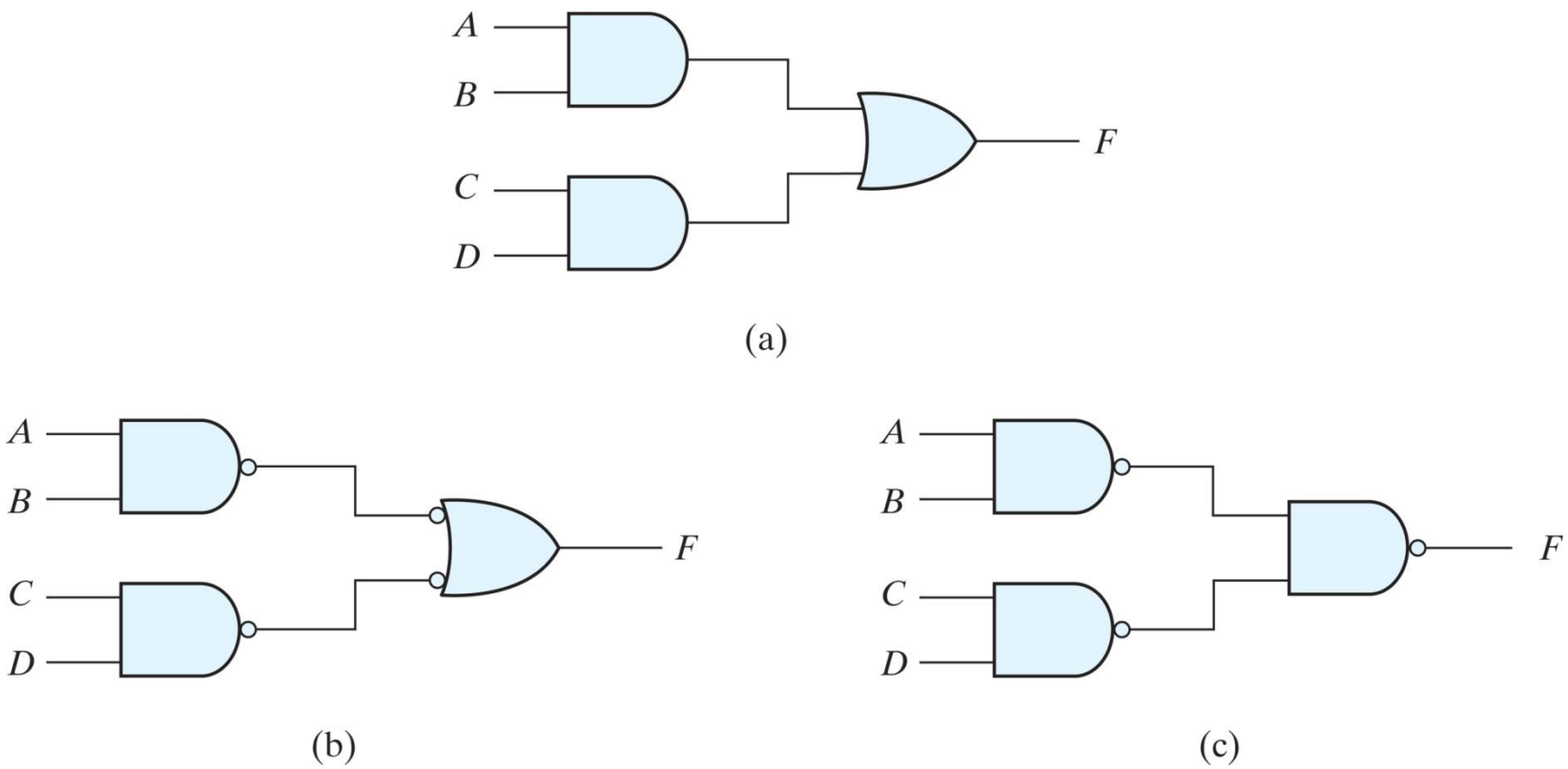
Alternate Symbols



Two graphic symbols for a three-input NAND gate

Two Level Implementation

- The implementation of Boolean functions with NAND gates requires that the functions be in sum-of-products form.

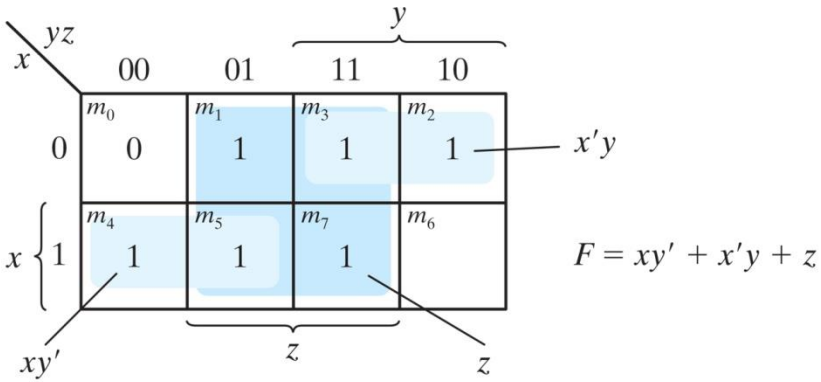


Three ways to implement $F = AB + CD$

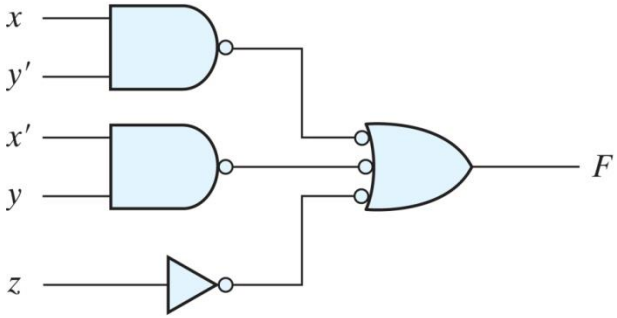
Example 20:

- Implement the following function with NAND gates.

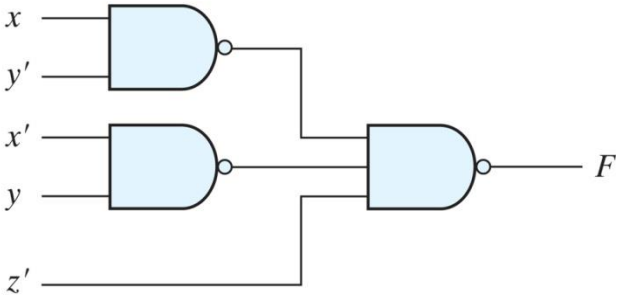
$$F(x, y, z) = \Sigma(1, 2, 3, 4, 5, 7)$$



(a)



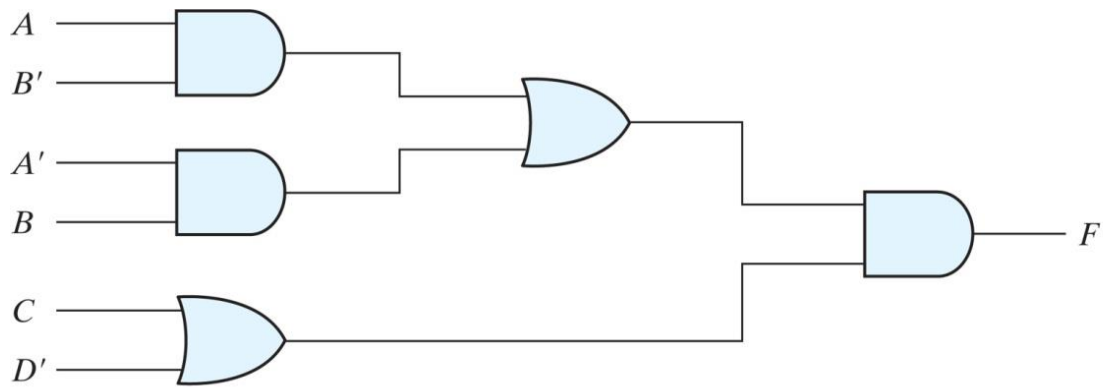
(b)



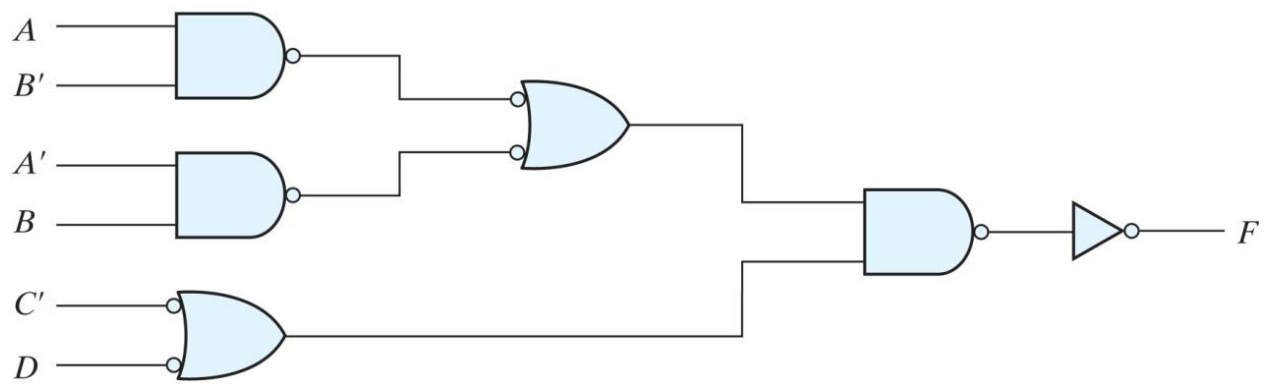
(c)

Multilevel NAND Circuits

- Implement $F = (AB' + A'B)\ (C + D')$



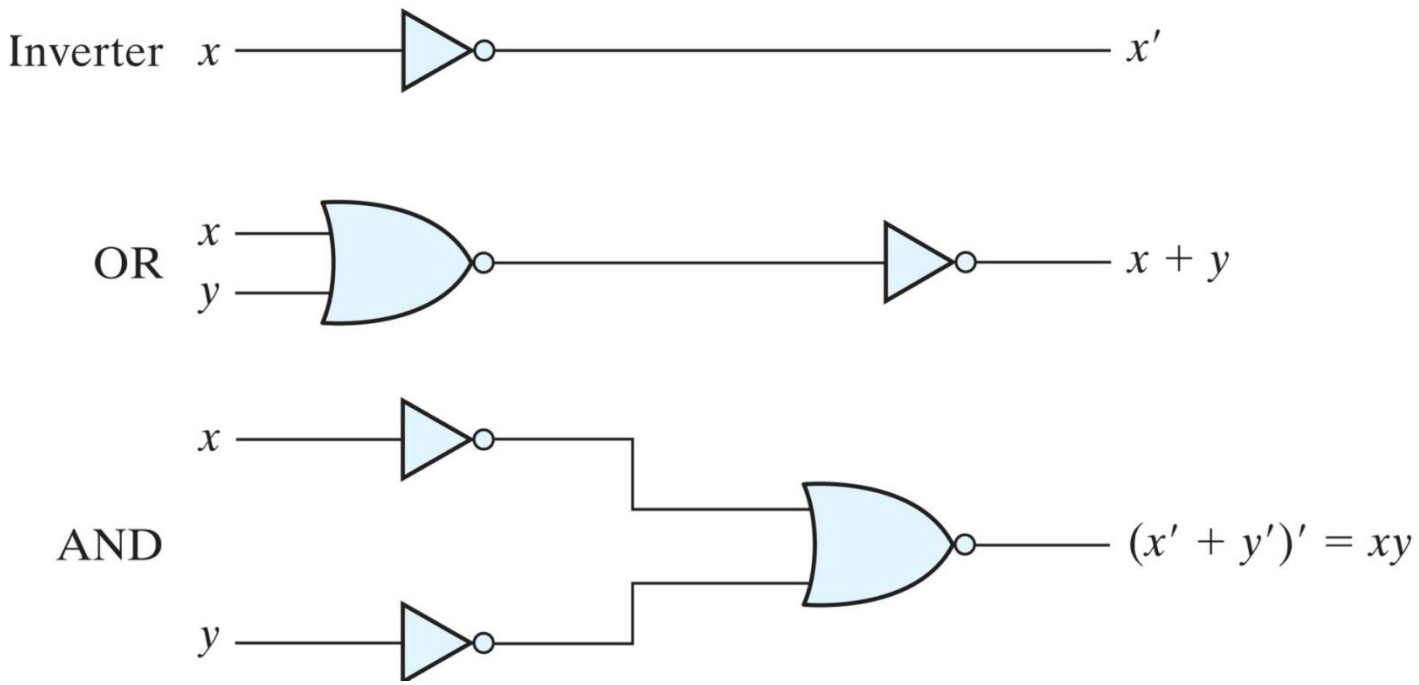
(a) AND–OR gates



(b) NAND gates

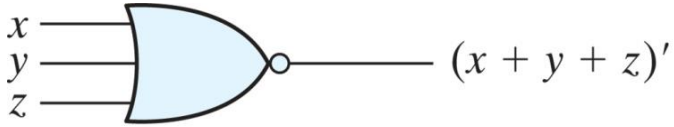
NOR implementation

- The NOR operation is the dual of the NAND operation.
  - All procedures and rules for NOR logic are the duals of the corresponding procedures and rules developed for NAND logic.

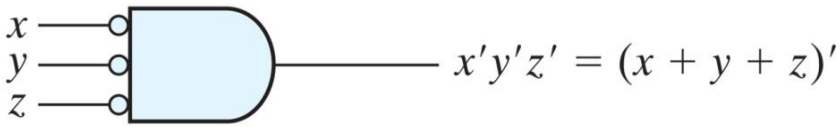


Logic operations with NOR gates

Alternate Graphic Symbols for NOR gate



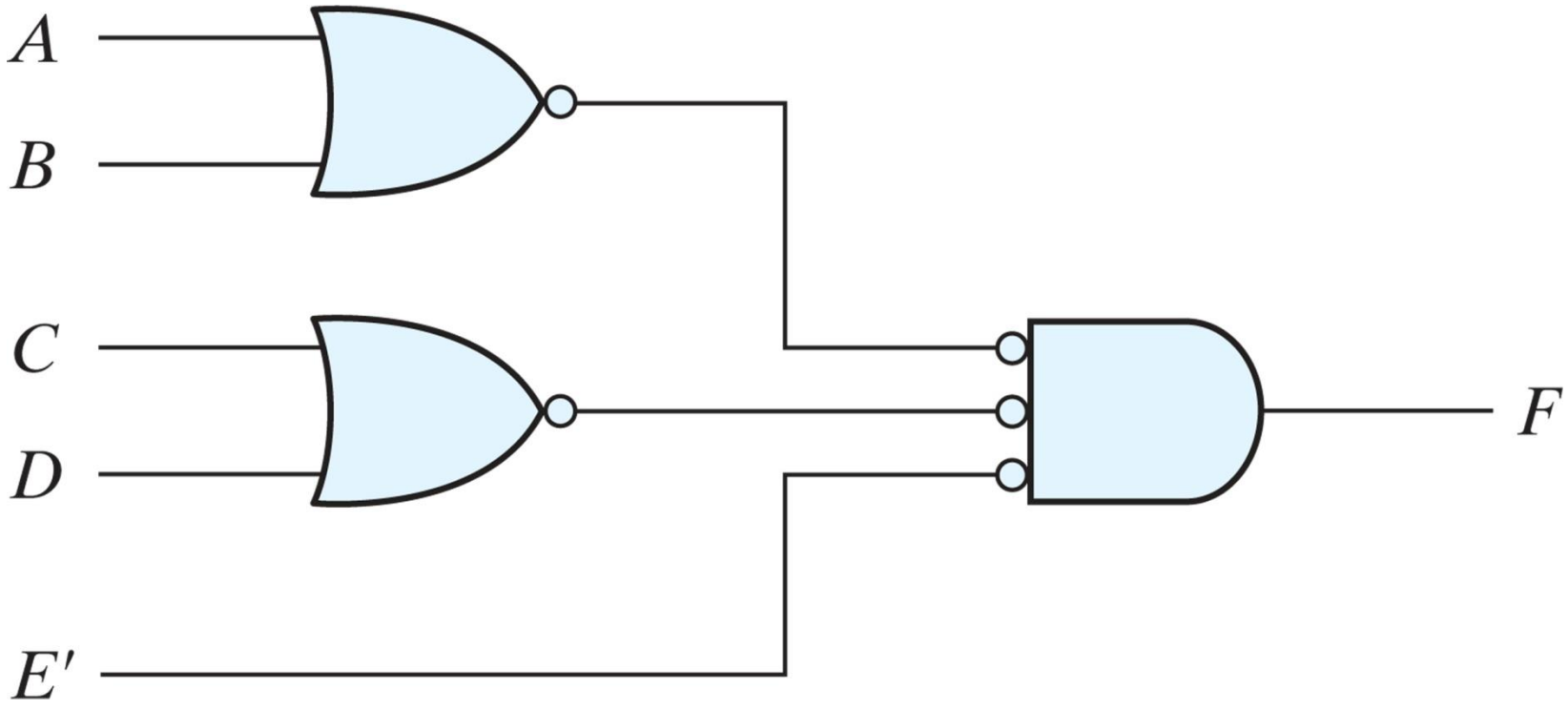
(a) OR-invert



(b) Invert-AND

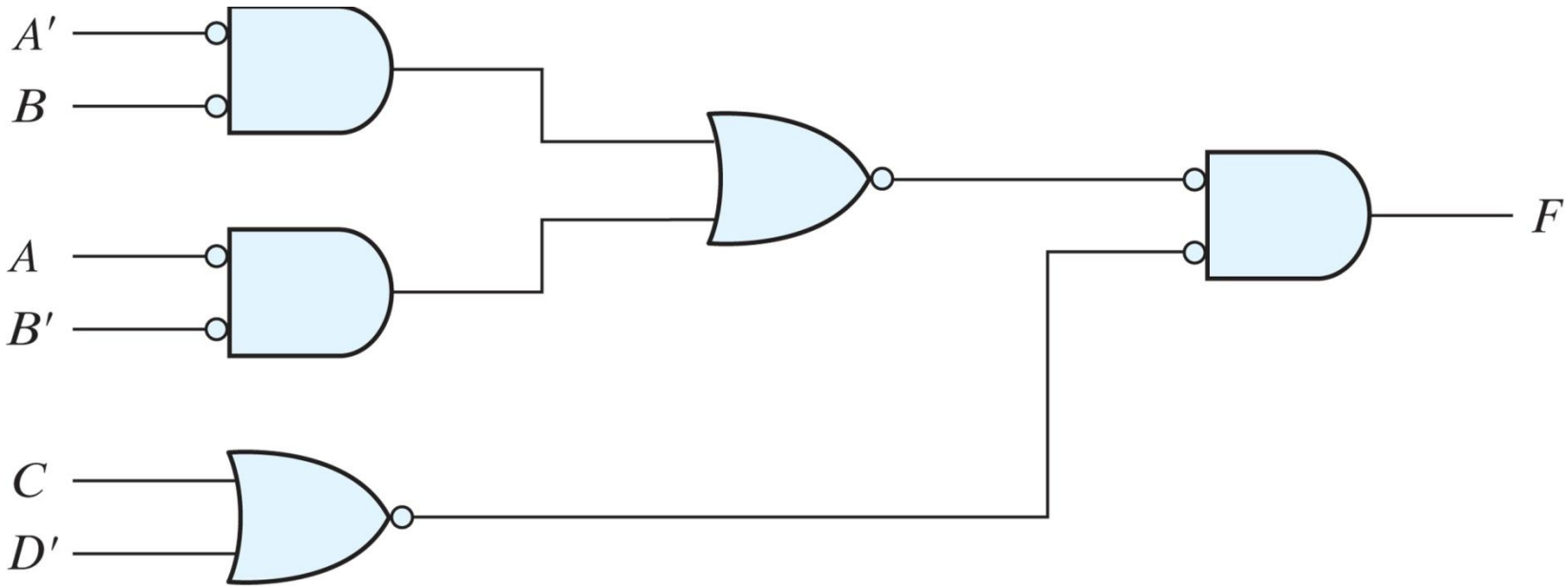
NOR implementation

- Implement F =(A+B)(C+D)E using NOR gates.



Multilevel NOR circuits

- Implement $F = (AB' + A'B)(C + D')$ with NOR gates.



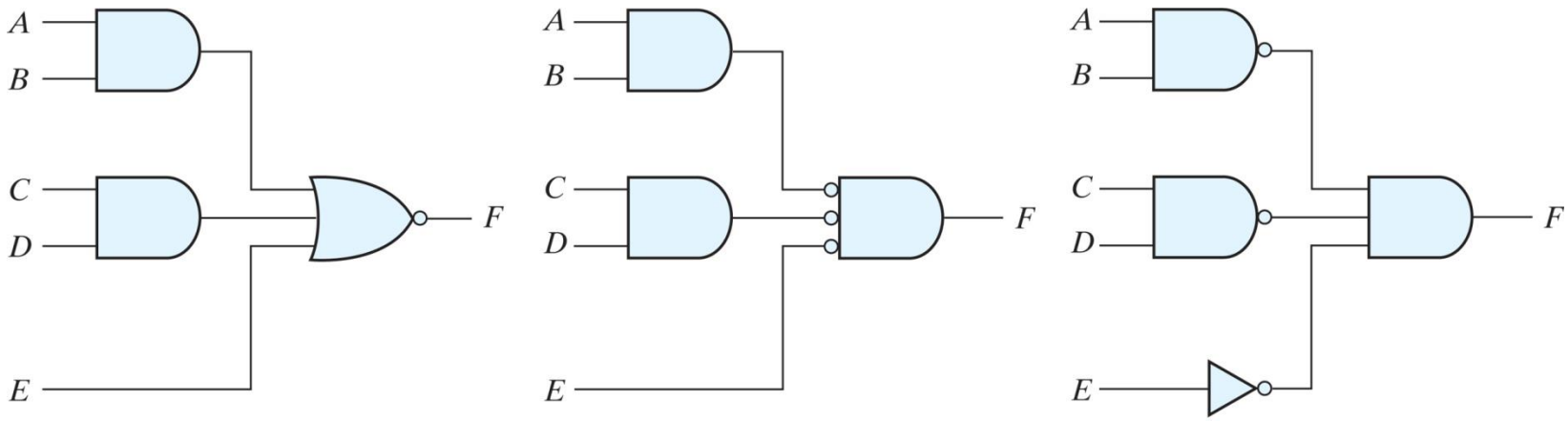
Other Two-Level Implementations

- *Nondegenerate* forms produce an implementation in sum-of-products form or products-of-sums form.

• AND-OR	OR-AND
• NAND-NAND	NOR-NOR
• NOR-OR	NAND-AND
• OR-AND	AND-NOR

AND-OR-INVERT Implementation

- Implement $F = (AB + CD + E)'$

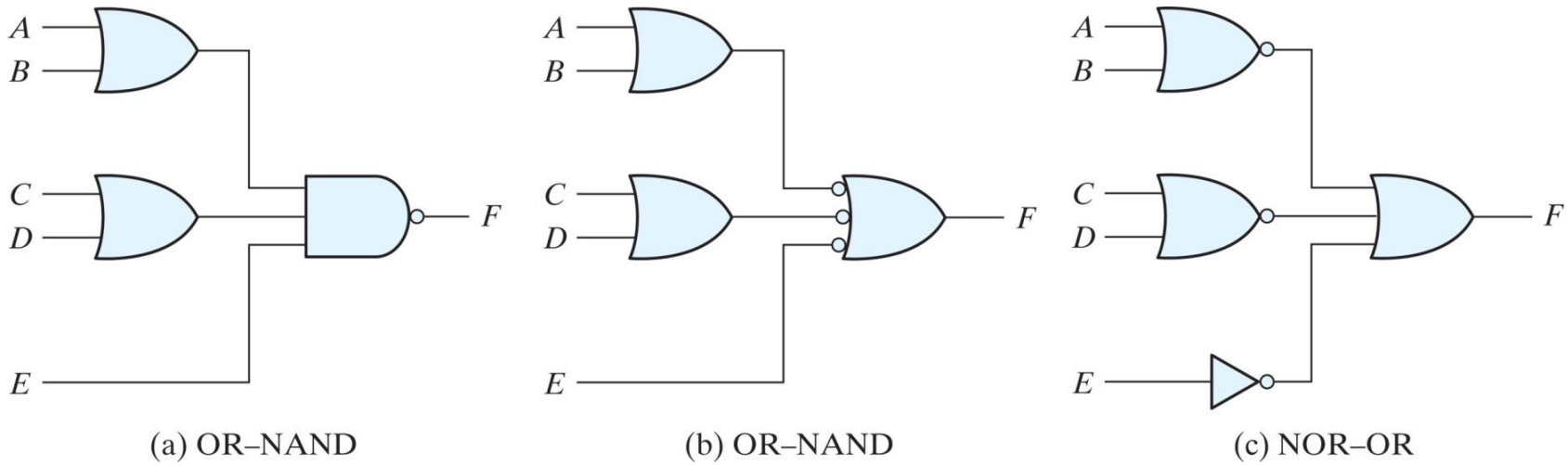


(a) AND–NOR (b) AND–NOR (c) NAND–AND

- The two forms, NAND-AND and AND-NOR, are equivalent. Both perform the AND-OR-INVERT function.

OR-AND-INVERT Implementation

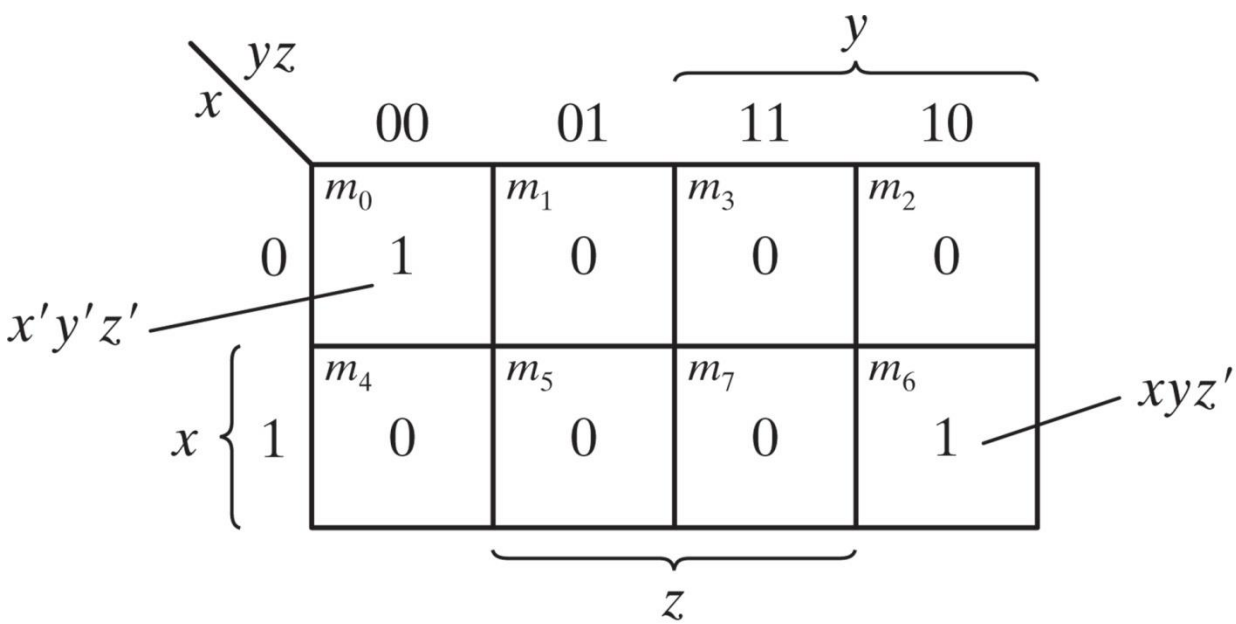
- Implement $F = [(A + B)(C + D)E]'$



- The OR-NAND and NOR-OR forms perform the OR-AND-INVERT function.

Example 21:

- Implement the function in the given K-map using a) AND-NOR, b) NAND-AND, c) OR-AND, and d) NOR-OR.

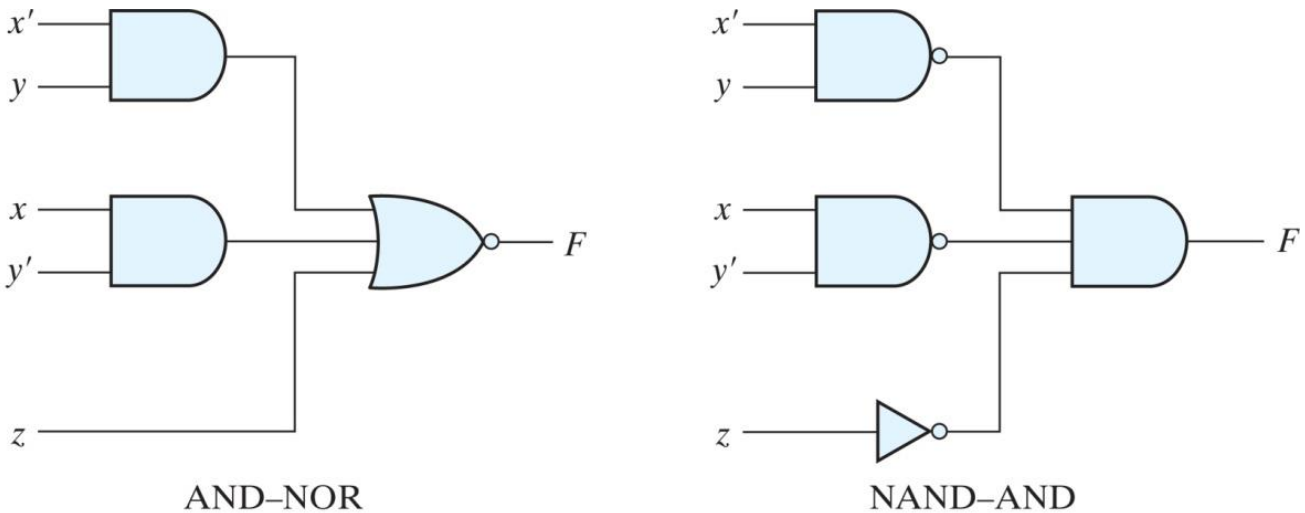


$$F = x'y'z' + xyz'$$

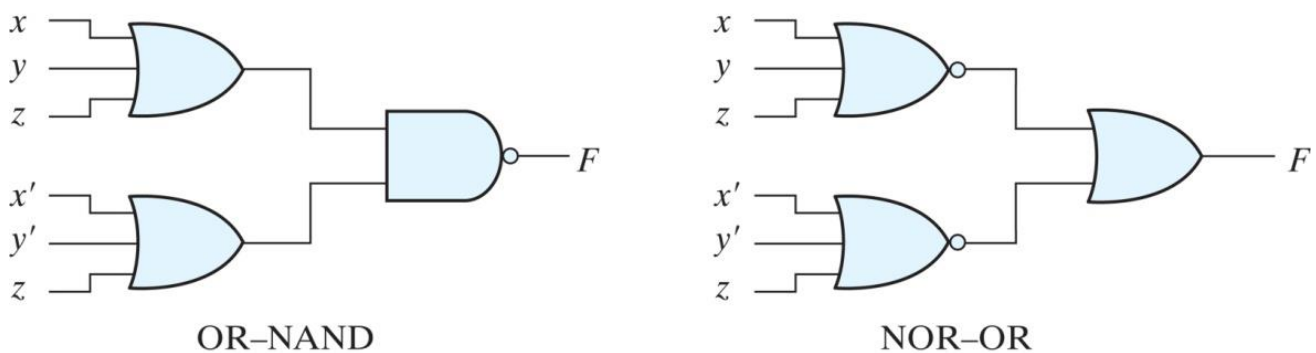
$$F' = x'y + xy' + z$$

(a) Map simplification in sum of products

Solution 21:



(b) $F = (x'y + xy' + z)'$



(c) $F = [(x + y + z)(x' + y' + z)]'$

Exclusive-OR Function

- The exclusive-OR function, denoted by symbol $\oplus$, is a logical operation that performs the following Boolean operation:

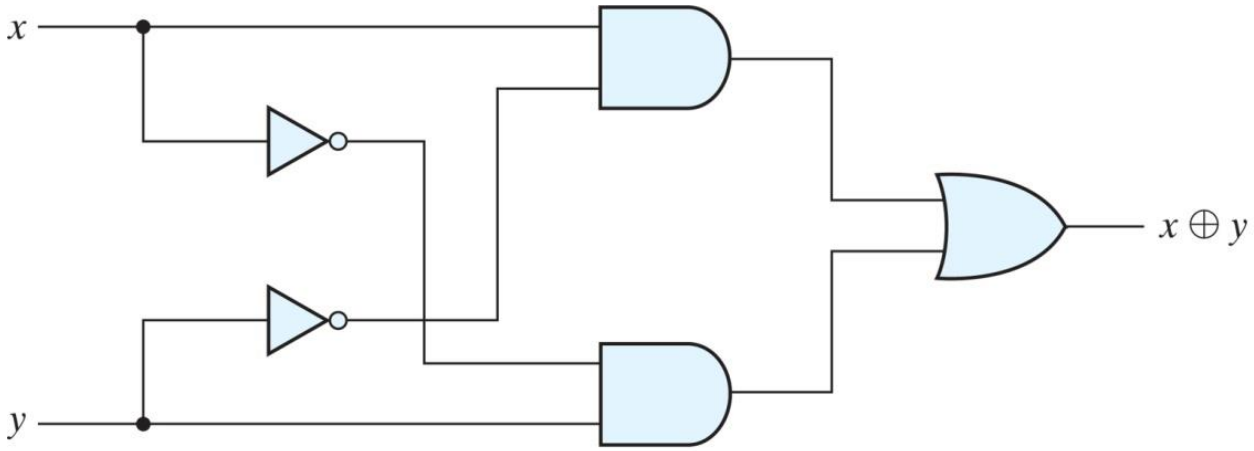
$$x \oplus y = xy' + x'y$$

- The following identities apply to the exclusive-OR operation:
  - $x \oplus 0 = x$
  - $x \oplus 1 = x'$
  - $x \oplus x = 0$
  - $x \oplus x' = 1$
  - $x \oplus y' = x' \oplus y = (x \oplus y)'$
- The exclusive-OR operation is both commutative and associative; that is,

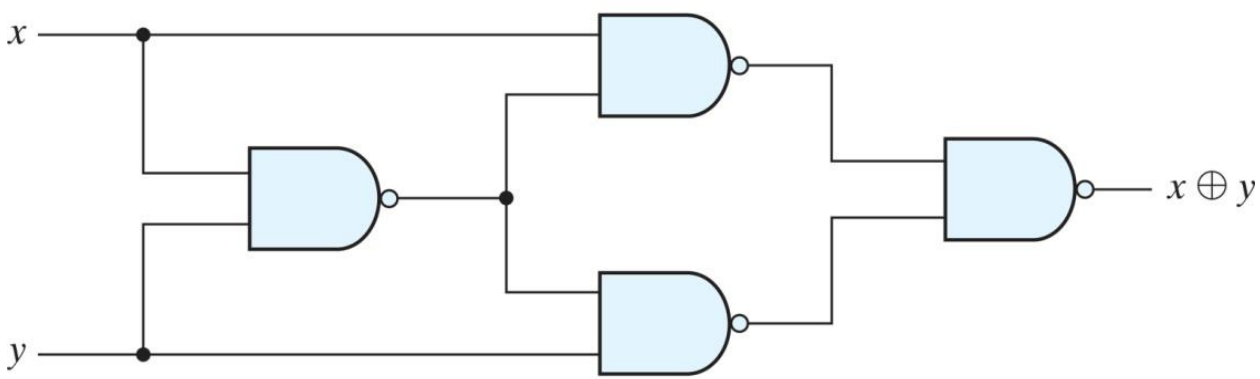
$$A \oplus B = B \oplus A$$

$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

Exclusive-OR implementations



(a) Exclusive-OR with AND–OR–NOT gates



(b) Exclusive-OR with NAND gates

Odd Function

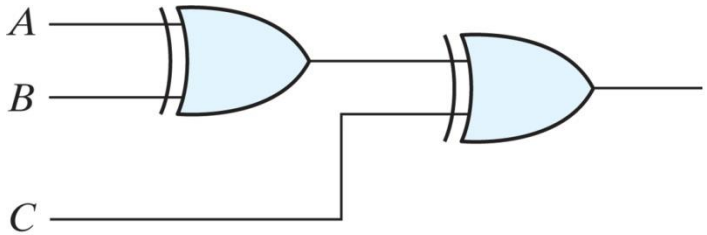
- A function where odd number of variables are equal to 1.

A \ BC	00	01	11	10
0	m_0	m_1 1	m_3	m_2 1
1	m_4 1	m_5	m_7 1	m_6

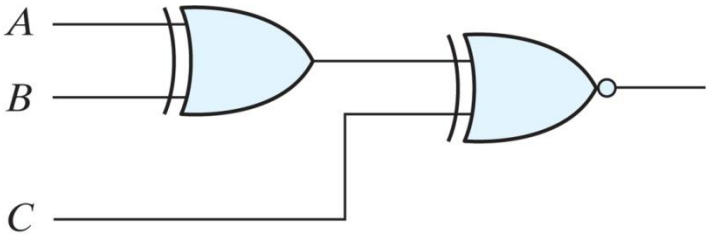
(a) Odd function $F = A \oplus B \oplus C$

A \ BC	00	01	11	10
0	m_0 1	m_1	m_3 1	m_2
1	m_4	m_5 1	m_7	m_6 1

(b) Even function $F = (A \oplus B \oplus C)'$



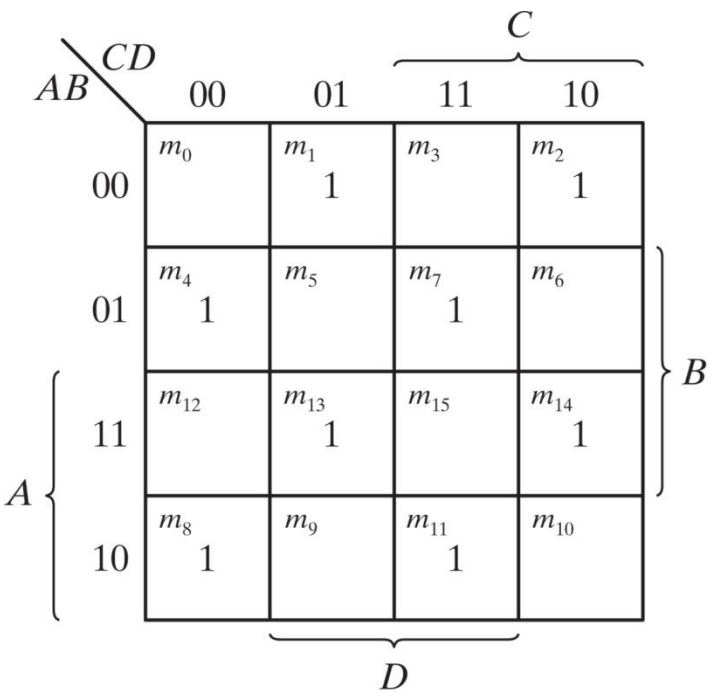
(a) 3-input odd function



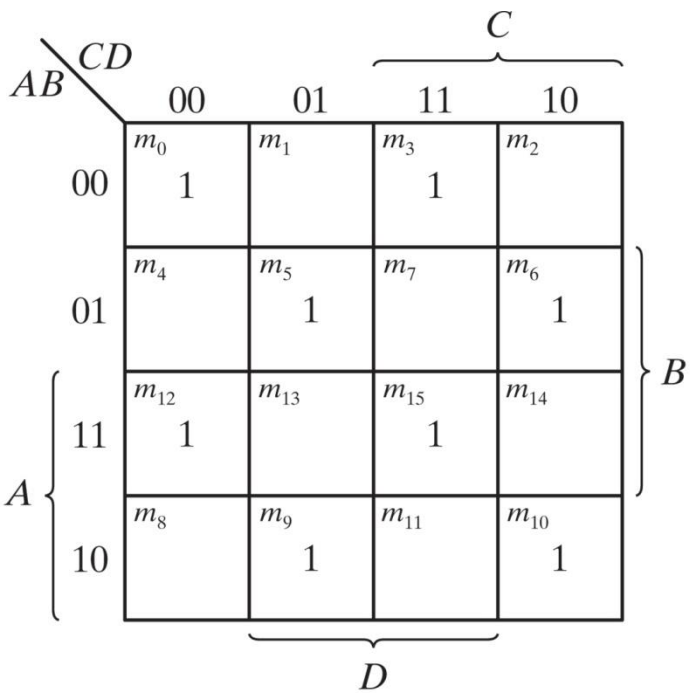
(b) 3-input even function

Logic diagram of odd and even functions

- In general, an *n*-variable exclusive-OR function is an odd function defined as the logical sum of the $2^n/2$ min-terms whose binary numerical values have an odd number of 1's.



(a) Odd function $F = A \oplus B \oplus C \oplus D$



(b) Even function $F = (A \oplus B \oplus C \oplus D)'$

Parity Generation and Checking

- A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even.

- The message, including parity bit is, is transmitted and then checked at the receiving end for errors.

- An error is detected if the checked parity does not correspond with the one transmitted.

- The circuit that generates the parity bit in the transmitter is called a *parity generator*. The circuit that checks the parity in the receiver is called a *parity checker*.

Even Parity Generator

Even-Parity-Generator Truth Table

Three-Bit Message			Parity Bit
x	*y*	*z*	*P*
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

- $P = x \oplus y \oplus z$

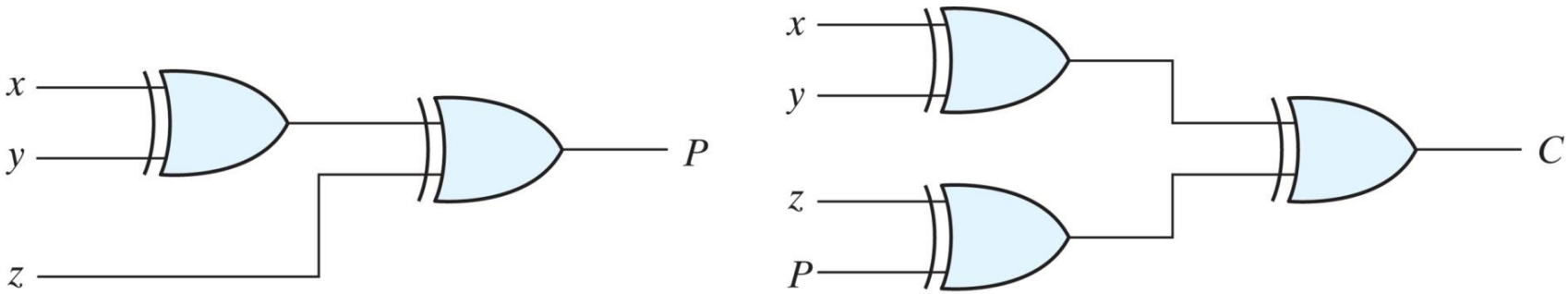
Even-Parity-Checker Truth Table

Even-Parity-Checker Truth Table

Four Bits Received				Parity Error Check
x	*y*	*z*	*P*	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

- $C = x \oplus y \oplus z \oplus P$

Logic diagram of a Parity Generator and Checker



(a) 3-bit even parity generator

(b) 4-bit even parity checker

Other Minimization Methods

- Tabulation Method
  - Quine-McCluskey Method

Thank you!!!